任务 1：数据库与查询条件基础说明

以 DVWA 登录功能 这个具体场景为例

1. 页面中的输入为什么会与数据库查询有关

登录页面的核心功能是验证用户身份，而用户的账号、密码信息都存储在后端的数据库中。当用户在登录框输入用户名和密码并提交时，后端程序需要从数据库中查找匹配的记录，判断用户是否合法。因此，用户的输入会被用来构造数据库查询语句，直接和数据库交互。

2. 一个最小业务链路是什么

用户输入账号密码 → 后端程序接收输入 → 构造 SQL 查询语句 → 向数据库发送查询请求 → 数据库执行查询并返回结果 → 后端根据结果判断是否登录成功 → 页面返回登录结果

3. 用户输入是如何进入查询条件的

用户输入的用户名和密码会直接拼接到 SQL 查询语句中，成为查询条件的一部分。

4. 为什么这为 SQL 注入创造了可能性

因为程序没有对用户输入做任何过滤或转义，就直接将其拼接进了 SQL 语句。这意味着用户输入的内容可以改变 SQL 语句原本的结构和逻辑。比如，用户在用户名输入框中输入 1’ OR 1=1 #，拼接后的 SQL 语句中后，这里的’闭合了原本的字符串引号，OR 1=1 让查询条件永远为真，#注释掉了后面的密码验证部分，最终程序会返回所有用户数据，导致登录绕过。

任务 2：DVWA SQL Injection Low 现象记录

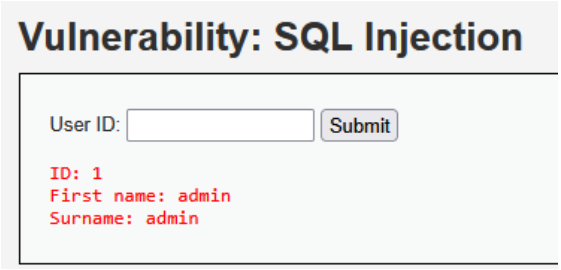
1. 实验页面或实验目标

实验页面：DVWA（安全级别设置为 Low）

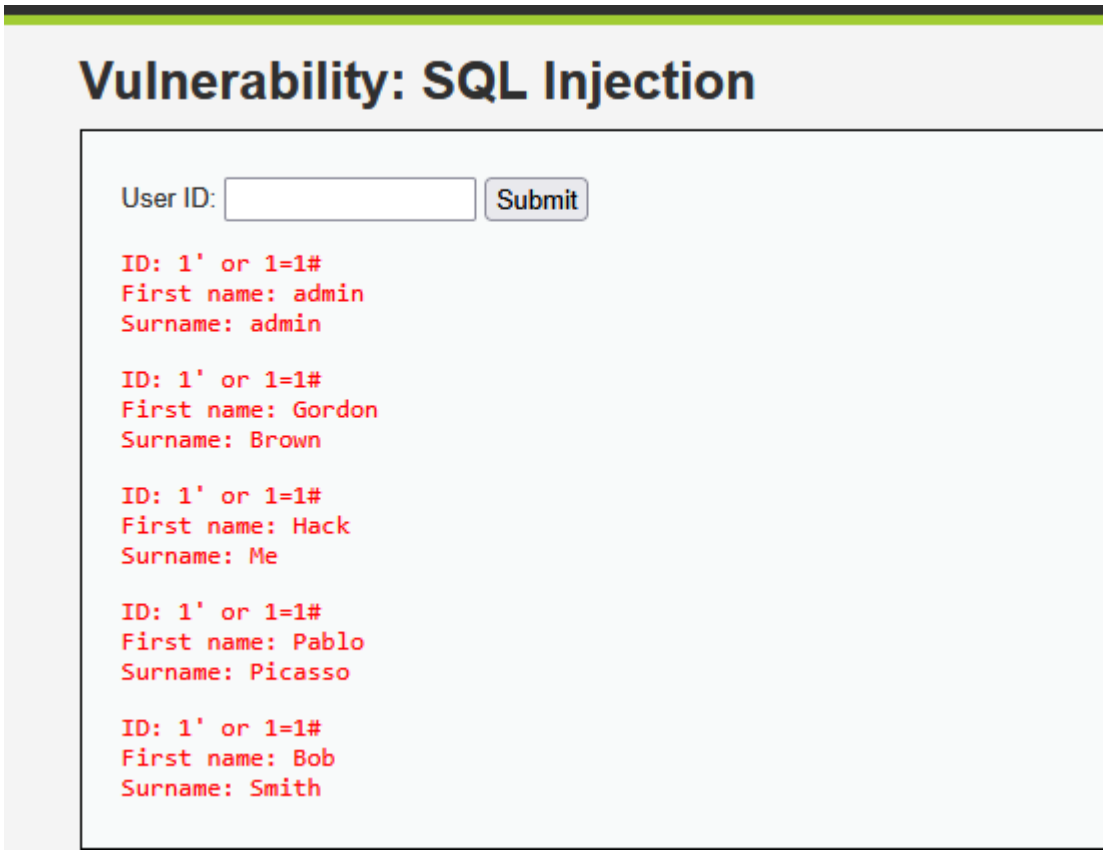
实验目标：观察普通输入与异常输入的返回差异，验证 差异的 SQL 注入本质

2. 普通输入与异常输入的对比

普通输入：1　　结果：查询 ID 为 1 的用户信息

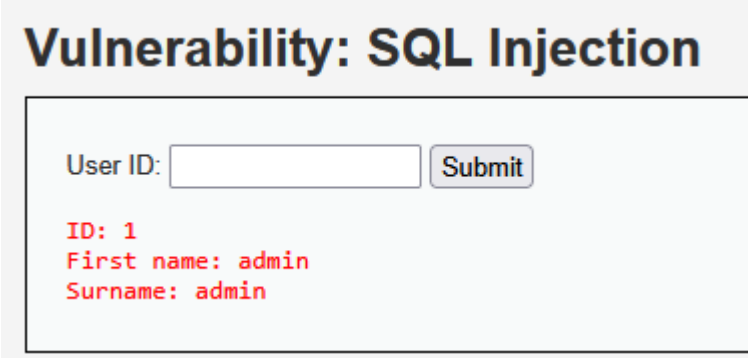


异常输入：1’ or 1=1#　　结果：构造恒成立条件，尝试查询所有用户信息



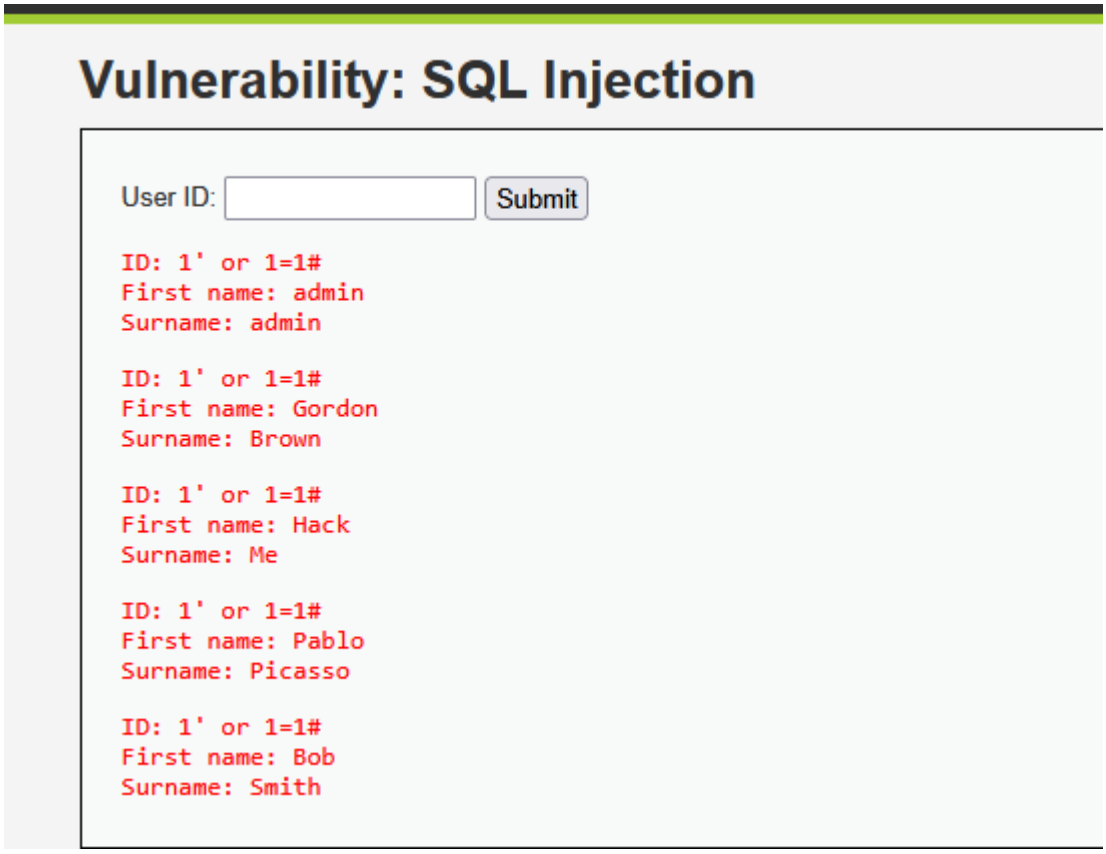
3. 返回结果的差异

普通输入结果返回：单条 ID 为 1 的数据



程序能根据输入 ID，从数据库中查询并返回对应记录，说明输入确实被用来构造 SQL 查询。

异常输入结果返回：数据库中所有用户的数据



用户输入的 1’ or 1=1#被成功拼接到 SQL 语句中，将查询条件修改为恒成立，程序执行了被篡改后的 SQL 逻辑，返回了所有数据，这是典型的 SQL 注入现象。

4. 为什么这种差异不像“正常查询变化”。

正常查询逻辑下，输入不同的 ID 只会返回对应的单条用户数据，输入不存在的 ID 只会返回 “找不到记录”，不会出现语法错误，也不会一次性返回全表数据。

而本次实验中，异常输入出现了输入恒成立条件时返回全表数据这样的不符合正常业务逻辑的现象，说明用户输入的逻辑被 SQL 引擎直接执行，完全改变了查询的条件和范围。

这种差异不是业务功能的正常表现，而是用户输入污染了 SQL 语句结构导致的异常行为，因此支撑了 “此处存在 SQL 注入漏洞” 的判断。

任务 3：机制分析与差异比较

1. 输入如何进入查询结构

用户输入 → 被程序直接拼接进SQL语句 → 数据库解析SQL语句 → 执行被篡改后的查询 → 页面回显结果或产生行为差异

以 DVWA Low 级别的 SQL 注入为例，用户输入$id没有经过任何过滤，直接嵌入到了 SQL 语句的 WHERE 条件中，成为了查询结构的一部分。

2. 为什么根因是“结构被污染”而不是“字符本身危险”

SQL 注入的核心，是用户输入改变了 SQL 语句的语法结构和执行逻辑，而不是某个字符本身有危险。

例如，在输入 1 时，SQL 语句正常执行，而输入 1’ or 1=1#时，SQL 语句结构被篡改，条件恒成立。这里的单引号’、OR 等字符本身不是 “危险字符”，

它们是 SQL 语法的一部分。真正的问题是程序把用户输入当成了 SQL 语句的一部分来解析，导致原本的查询逻辑被破坏。

3. Low 与 Medium 的差异在哪里

维度	Low 级别	Medium 级别
输入获取方式	直接通过$_GET['id']获取	通过 mysql_real_escape_string()对输入进行转义，同时限制为数字类型
处理逻辑	无任何过滤或转义，直接拼接	对单引号、双引号、反斜杠等特殊字符进行转义，且输入以数字形式处理
注入难度	极易，直接拼接字符串即可	需绕过转义
本质差异	完全没有对输入进行处理，查询结构完全暴露	对输入做了简单的转义和类型限制，试图阻止结构被污染

在 DVWA 中，Low 和 Medium 级别的核心差异在于对用户输入的过滤处理方式

4. 普通 SQL 注入与 Blind SQL Injection 的差异在哪里

维度	普通 SQL 注入	Blind SQL Injection
回显特征	数据库查询结果会直接在页面回显	页面不直接返回数据，只返回 “正常/错误/延时” 等状态
信息获取方式	直接通过回显的内容获取数据	通过页面的响应差异（布尔判断、时间延迟）推断数据
典型场景	页面会显示查询到的用户信息、错误信息	页面只显示 “登录成功或失败”，或响应时间存在差异
攻击效率	高，一次注入即可看到结果	低，需要逐位猜解数据

5. 为什么盲注和普通注入根因相同，但证据形式不同

根因相同：两者的本质都是用户输入污染了 SQL 查询结构，导致原本的查询逻辑被篡改，数据库执行了攻击者构造的恶意 SQL 语句。

证据形式不同：

普通注入：页面直接回显查询结果或错误信息，证据是 “返回的内容异常”。

盲注：页面不回显数据，只通过 “响应状态或响应时间” 体现差异，证据是 “行为差异”。

强制判断题

1. SQL 注入的根因更接近：B. 输入进入了不该进入的查询结构

理由：SQL 注入的本质是用户输入被当作 SQL 语句的一部分解析，污染了查询结构，而不是因为输入里有几个 “危险字符”。字符本身只是语法符号，问题出在程序对输入的处理方式上。

2. Blind SQL Injection 和普通 SQL 注入的最大差异更接近：B. 证据形式不同

理由：两者的根因都是查询结构被污染，只是应用层的回显机制不同，导致漏洞的表现形式不同：普通注入靠回显内容判断，盲注靠行为差异判断。

任务 4：SQL 注入防护清单与解释

1. 参数化查询

解决什么问题：从根本上阻止用户输入污染 SQL 查询结构。它会将 SQL 语句的逻辑部分与用户输入的数据部分完全分离，数据库会先解析预编译的 SQL 模板，再将用户输入作为独立参数传入，输入永远不会被解析为 SQL 语法的一部分。

控制哪一层：根因控制层

它不能替代什么：不能替代错误处理、日志审计等 “后果控制” 措施，也不能弥补业务逻辑本身的漏洞。

2. 输入校验

解决什么问题：在应用层提前拦截不符合业务预期的恶意输入。例如对 ID 类输入强制校验为数字、对用户名限制合法字符集与长度，减少恶意 payload 进入后端的机会。

控制哪一层：暴露面控制层

它不能替代什么：不能替代参数化查询。输入校验的规则容易被绕过，且无法覆盖所有业务场景的输入格式，不是根本性防护手段。

3. 错误处理

解决什么问题：避免数据库错误信息直接暴露给攻击者，防止攻击者通过错误回显推断数据库结构，减少盲注的可利用信息。

控制哪一层：暴露面控制层

它不能替代什么：不能修复 SQL 注入漏洞本身，只是隐藏了攻击证据，攻击者仍可通过时间盲注、布尔盲注等方式继续攻击。

4. 最小权限

解决什么问题：限制数据库连接账户的权限，即使 SQL 注入发生，攻击者也无法执行高危操作，从而缩小攻击造成的后果范围。

控制哪一层：后果范围控制层

它不能替代什么：不能阻止注入攻击的发生，只是限制了攻击成功后的破坏力。

5. 可补充其他合理措施

Web 应用防火墙

解决什么问题：在应用前端识别并拦截常见的 SQL 注入攻击 payload，如包含 OR、UNION、--等关键字的请求，作为额外的防护层。

控制哪一层：暴露面控制层

它不能替代什么：不能替代参数化查询这类根本性防护。WAF 的规则存在被绕过的可能，也无法防御零日注入漏洞。

收口项 1：SQL 注入因果链/防护关系图

环节	内容	对应因果关系	防护控制点
1	输入点	用户输入的内容是整个注入攻击的起点	输入校验：对输入做格式、长度、字符集限制，拦截恶意 Payload

环节	内容	对应因果关系	防护控制点
2	查询构造	程序将用户输入直接拼接到 SQL 语句中，未做处理	参数化查询：预编译 SQL 模板，将用户输入作为参数传入，从根源阻止结构污染
3	数据库解析/执行	数据库将拼接后的恶意 SQL 当作合法语句执行	最小权限：限制数据库账户权限，即使注入成功也无法执行高危操作
4	结果差异或行为证据	页面回显错误信息、全表数据，或出现响应延迟、布尔状态差异	错误处理：关闭详细数据库错误回显，隐藏攻击线索
5	根因判断	核心问题是用户输入污染了 SQL 查询结构，而非字符本身危险	

收口项 2：学生自评表

1. 我本章最清楚的一点是什么？

SQL 注入的核心根因是用户输入污染了 SQL 查询结构，而不是输入里的 “危险字符” 本身。参数化查询是从根源解决注入问题的关键手段，其他防护措施都只是补充或缓解手段。

2. 我最容易混淆的一点是什么？

普通 SQL 注入与盲注的差异。一开始误以为两者的根因不同，后来才理解它们的底层漏洞都是查询结构被污染，只是应用层的回显机制不同，导致漏洞的表现形式不一样。

3. 我本次作业中哪一段最需要教师复核？

关于 Low 与 Medium 级别的 DVWA SQL 注入差异分析，以及不同防护措施对应的控制层级划分，担心自己对层级的理解不够准确。

4. 如果重做一次，我会优先改哪一部分？

会优先优化收口项 1 的因果链关系图，用更直观的流程图形式呈现，而不是文字表格，让因果关系和防护对应更清晰；同时补充更多具体的 Payload 示例来支撑分析。

收口项 3：AI 输出审核与人工复核记录

1. 本作业哪些内容使用过 AI 辅助

任务 1-4 的基础框架梳理、关键概念解释、SQL 注入因果链整理、防护措施的层级划分，均使用了 AI 辅助生成初稿。

2. 哪些判断是你自己复核后保留的

SQL 注入的根因是 “查询结构被污染” 而非字符本身危险
普通注入与盲注的根因相同，差异仅在于证据形式
DVWA Low 与 Medium 级别的核心差异是输入过滤方式不同

3. 哪些内容你发现 AI 输出不准确并进行了修正

修正了 AI 对盲注的定义偏差，补充了 “盲注依然依赖查询结构被污染” 的底层逻辑，避免了 “盲注根因不同” 的错误理解

4. 你最终如何确认结论和证据是一一对应的

结合数据库执行原理，梳理了输入拼接→结构污染→执行恶意语句的完整链路，验证了根因判断的合理性